

3. Repository structure

I would use a normal Python package with a `src` layout:

```
ek80_fm_pipeline/
├── pyproject.toml
├── README.md
├── LICENSE
├── .gitignore
├── src/
│   └── ek80_fm/
│       ├── __init__.py
│       ├── io.py
│       ├── navigation.py
│       ├── calibration.py
│       ├── preprocessing.py
│       ├── boundaries.py
│       ├── angles.py
│       ├── single_target.py
│       ├── target_properties.py
│       ├── export.py
│       └── pipeline.py
├── config/
│   └── default.yaml
├── scripts/
│   └── run_pipeline.py
├── notebooks/
│   └── development/
└── tests/
    ├── unit/
    ├── integration/
    └── fixtures/
```

I would **not** put everything into a single `processing.py`.

The functions should be individually usable, for example:

```
from ek80_fm.navigation import add_navigation
from ek80_fm.boundaries import detect_surface
from ek80_fm.single_target import detect_fm_targets
```

That becomes particularly useful when you start experimenting interactively in notebooks.

4. Dependency strategy

This is the part where I would deviate somewhat from your original idea.

Stable dependency

Eventually:

```
dependencies = [  
    "echopype==0.11.1",  
    "numpy",  
    "xarray",  
    "pandas",  
    "scipy",  
]
```

But **for the FM development phase**, I would pin the echopype PR branch to an exact commit rather than trying to reconstruct its FM calibration code piecemeal.

The current PR branch contains many changes beyond the original single-target detector: FM calibration, FM TS processing, beam correction, target-detection changes, etc. The branch's latest listed commit is `b26d049` from 22 September 2026.

So during development I would use something conceptually like:

```
echopype @ git+https://github.com/LOCEANlloydizard/echopype.git@b26d049
```

with the exact commit pinned in your lock file.

Then later, once the relevant echopype functionality has been released, you switch back to:

```
echopype==<released-version>
```

This gives you:

today: reproducible access to the FM development code

later: normal PyPI dependency

without changing your pipeline API.

I would explicitly isolate that dependency in one module so that the transition later is easy.

Step 2 — SQLite navigation

I would **not** try to stuff the SQLite data into `EchoData.Platform` initially.

Instead create:

```
nav = read_navigation(db_path, start_time, end_time)
```

returning an xarray dataset:

```
ping_time  
latitude  
longitude
```

Then:

```
ds = add_navigation(ds, nav)
```

The navigation function should:

1. read the SQLite observations;
2. convert all timestamps to one explicitly defined datetime representation;
3. sort by time;
4. remove duplicate timestamps;
5. interpolate latitude and longitude onto acoustic `ping_time`;
6. return an xarray dataset.

Conceptually:

```
SQLite GPS  
  ↓  
pandas DataFrame  
  ↓  
xarray Dataset  
  ↓  
.interp(ping_time=...)  
  ↓  
latitude / longitude aligned with acoustic pings
```

This completely avoids the failure mode in issue #1493.

Also: **do not silently extrapolate outside the GPS time range**. Missing navigation should normally become `NaN`, with a warning or quality flag.

A useful output would be:

```
latitude  
longitude  
navigation_valid
```

where `navigation_valid` records whether interpolation was actually possible.

The existing echopype implementation itself uses linear interpolation to align GPS to acoustic `ping_time`; it also now handles duplicate location timestamps.

Step 3 — pulse compression

This deserves more care than it initially appears.

Echotype already contains `compress_pulse()`, and it is implemented using xarray/Dask-compatible processing.

However, in the released code the surrounding calibration machinery is still somewhat BB-centric. For example, the released `add_splitbeam_angle()` documents pulse compression as valid for "BB" complex data, while FM support was introduced separately in calibration.

The PR branch addresses this much more directly. Its FM-specific calibration path explicitly does:

```
FM complex samples
    ↓
matched filter
    ↓
pulse compression
    ↓
calibrated Sp / TS(f)
```

and it calls a pulse-compressed-signal function internally.

My recommendation

Do **not** make "pulse compression" a public function that merely wraps a private echotype implementation.

Instead define your own abstraction:

```
pc = pulse_compress_fm(ed, channel=...)
```

with a documented return structure.

For example:

```
pulse_compressed
  dimensions:
    ping_time
    beam
    range_sample

  variables:
    complex_signal
    echo_range
```

That becomes the canonical intermediate representation for the entire rest of the pipeline.

This is especially important because you don't want to pulse-compress the same RAW signal three separate times just because Sv, split-beam angle and target detection were implemented independently.

Step 4 — seafloor detection

This one should mostly remain echopype.

Current echopype exposes:

```
ep.mask.detect_seafloor(...)
```

with "basic" and "blackwell" methods. The result is a one-dimensional bottom line indexed by `ping_time`.

I would initially implement:

```
bottom = detect_bottom(ds_sv, config)
```

where your wrapper handles:

```
channel selection
parameters
NaN handling
bottom → depth/range alignment
```

The important thing is that the output becomes part of the common pipeline representation:

```
bottom_depth(ping_time)
```

Then target detection can use:

```
target_depth < bottom_depth - bottom_offset
```

The target detector in PR #1588 already supports a bottom constraint.

Step 5 — surface turbidity

I would definitely rewrite `find_waves.py`.

Here it is: https://github.com/BalticSeabird/echoedge/blob/new-structure/lib/find_waves.py

The current implementation is a small NumPy algorithm which:

- replaces NaNs;
- looks at each ping independently;
- searches for consecutive samples below a threshold;
- returns a wave/surface line;
- modifies the supplied array in place.

That last property is undesirable in a modern xarray pipeline.

The new version should have an API resembling echotype:

```
surface = detect_surface(  
    ds_sv,  
    method="threshold",  
    params={  
        "threshold": -50,  
        "n_consecutive": 5,  
        "beam_dead_zone": ...,  
    },  
)
```

and return:

```
surface_range(ping_time)
```

or preferably:

```
surface_depth(ping_time)  
surface_range(ping_time)
```

Important point

I would initially make the **new implementation numerically equivalent to the current `find_waves.py`**, rather than improving the algorithm at the same time.

That gives you a very clean development sequence:

```
old implementation  
    ↓  
reference test cases  
    ↓  
xarray implementation  
    ↓  
must produce same answer  
    ↓  
only then improve algorithm
```

This prevents algorithmic changes and software-engineering changes becoming impossible to disentangle.

For a first version I would implement the per-ping algorithm using `xr.apply_ufunc()` so that it remains compatible with xarray and, potentially, Dask.

Configuration should live outside the code

Don't bury thresholds in Python.

For example:

```
input:
  sonar_model: EK80
  waveform_mode: FM
  encode_mode: complex

navigation:
  interpolation: linear
  max_gap_seconds: 5

bottom:
  method: basic
  threshold_db: -50
  offset_m: 0.5

surface:
  method: threshold
  threshold_db: -50
  consecutive_samples: 5

single_target:
  method: echoview_method_2
  pldl_db: 6
  min_norm_pulse: 0.8
  max_norm_pulse: 1.5
  max_beam_comp_db: 6
  max_sd_minor_deg: ...
  max_sd_major_deg: ...

output:
  csv: true
```

This will be extremely useful when you later process different surveys or transducers.